

Aula 6 - Shell Avançado II - Bash como Linguagem de Programação

Nas aulas anteriores, aprendemos a navegar pelo sistema de arquivos (Aula 2), conectar em máquinas remotas via SSH (Aula 3), dominar permissões e links simbólicos (Aula 4) e entender canais de comunicação, redirecionamento, pipes, processos e threads (Aula 5). Em todas essas aulas, cada comando foi digitado manualmente, um por um. Mas e se você precisar executar a mesma sequência de 15 comandos todos os dias para verificar se os robôs do laboratório estão funcionando? E se precisar processar centenas de arquivos de log automaticamente toda manhã?

É aqui que o Bash deixa de ser apenas um terminal interativo e se revela como uma **linguagem de programação completa**. Assim como Python, o Bash possui variáveis, tipos de dados, operadores, condicionais (`if/else`), loops (`for, while`), funções e arrays. A diferença é que o Bash é **nativo do sistema operacional**: não precisa ser instalado e orquestra diretamente todas as ferramentas do Linux de uma forma que nenhuma outra linguagem consegue tão naturalmente.

1. Linguagem Bash

Até aqui, você aprendeu a usar o terminal de forma interativa: digitar um comando, ver o resultado, digitar outro. Isso é poderoso, mas é como cozinhar seguindo uma receita passo a passo toda vez — funciona, porém é lento e sujeito a erros. Agora, vamos aprender a **escrever a receita inteira em um arquivo** para que o computador a execute sozinho, quantas vezes quisermos, sem erro.

A partir desta seção, tratamos o Bash como uma **linguagem de programação**. Se você já programa em Python, verá muitas semelhanças: variáveis, condicionais, loops e funções existem nos dois. A diferença é que o Bash foi feito para orquestrar **comandos do sistema** — tudo que aprendemos nas Aulas 2 a 5 pode ser empacotado em scripts automatizados.

1.1 Script

Um **Script Bash** nada mais é do que um arquivo de texto contendo uma sequência de comandos que o shell executa um por um, exatamente como se você os digitasse no terminal. A diferença é que, em vez de digitar cada comando manualmente, você os escreve uma vez no arquivo e pode executar tudo com um único comando. É assim que transformamos tarefas repetitivas em automações confiáveis.

1.1.1 O shebang

A primeira linha de qualquer script bem escrito é o chamado shebang. O nome vem da contração de hash (`#`) com bang (`!`), os dois primeiros caracteres da linha. Ela instrui o sistema operacional sobre qual programa deve ser usado para interpretar o restante do arquivo. Sem ela, o sistema pode tentar executar o script com o interpretador padrão do terminal atual, que pode variar de máquina para máquina. Com ela, o comportamento do script se torna previsível em qualquer ambiente.

O shebang não é exclusivo do Bash. Qualquer linguagem interpretada pode usá-lo, bastando apontar para o executável correto:

Linguagem	Shebang
Bash	<code>#!/bin/bash</code>
Python	<code>#!/usr/bin/env python3</code>
Ruby	<code>#!/usr/bin/env ruby</code>
Perl	<code>#!/usr/bin/env perl</code>
Node.js	<code>#!/usr/bin/env node</code>

O padrão `#!/usr/bin/env` usado nas demais linguagens é uma forma mais portátil de localizar o interpretador. Em vez de fixar um caminho absoluto, ele pede ao sistema que encontre o executável no PATH — a lista de diretórios onde o Linux procura por programas. No caso do Bash, usar `/bin/bash` diretamente é a convenção mais comum porque esse caminho é praticamente universal em sistemas Linux.

Abra o nano e crie o primeiro script dentro da pasta `scripts` do nosso projeto:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ nano scripts/ola.sh
```

Dentro do editor, escreva o seguinte conteúdo:

```
#!/bin/bash

# Isto é um comentário, ignorado pelo interpretador
echo "Olá, mundo!" # Comentário inline também funciona
```

Salve com `Ctrl+O` e saia com `Ctrl+X`. Leia a linha `#!/bin/bash` assim: "use o programa localizado em `/bin/bash` para interpretar tudo que vier abaixo". O `#!` é o sinal que o kernel reconhece como essa instrução. Todo o restante da linha é o caminho do interpretador.

Você deve ter notado o `echo` dentro do script. Anteriormente, usamos bastante esse comando para combinar com redirecionamentos, onde o descrevemos como o equivalente do `print()` do Python: ele simplesmente imprime na tela o texto que recebe. Dentro de um script, ele cumpre exatamente o mesmo papel, servindo como a principal forma de exibir mensagens, resultados e informações para quem executa o programa.

1.1.2 Criando e executando um script

Com o arquivo criado, temos duas formas de executá-lo:

- Forma 1: passar o arquivo diretamente ao interpretador

```
bash scripts/ola.sh
```

- Forma 2: conceder permissão de execução e chamar diretamente

```
chmod +x scripts/ola.sh
./scripts/ola.sh
```

O `./` antes do nome do script é obrigatório na segunda forma porque o diretório atual não faz parte do PATH do sistema, então o shell não encontraria o arquivo pelo nome sozinho. Já na primeira forma, você chama o `bash` explicitamente e passa o arquivo como argumento, o que dispensa tanto a permissão de execução quanto o shebang.

1.1.3 Formatação de Texto

Ainda sobre o `echo`, anteriormente vimos que ele funciona como o `print()` do Python, imprimindo texto na tela. Mas quando usamos a flag `-e`, abrimos acesso a um conjunto de sequências especiais que controlam a formatação da saída. Dentro de scripts, isso é o que separa um resultado legível de um amontoado de texto sem estrutura.

As sequências mais usadas controlam o espaçamento e a quebra do texto:

Sequência	Efeito
<code>\n</code>	Insere uma quebra de linha
<code>\t</code>	Insere uma tabulação horizontal, útil para alinhar colunas

Por exemplo:

```
#!/bin/bash

echo -e "Status do sistema:\nTodos os robôs operando" # \n quebra em duas
linhas
echo -e "Robo\tBateria\t\tStatus" # \t alinha como
colunas
echo -e "roboA\t87%\t\tOnline"
echo -e "roboB\t43%\t\tOnline"
```

Salve com `Ctrl+O` e saia com `Ctrl+X`. Ao executar, o primeiro `echo` quebra a string em duas linhas e o segundo alinha as palavras em colunas usando tabulação. Teremos a saída abaixo:

```
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$ bash
scripts/formatos.sh
Status do sistema:
Todosos os robôs operando
Robo    Bateria Status
robo A  87% Online
robo B  43% Online
```

Cores com sequências ANSI

O `-e` também abre acesso às **sequências de escape ANSI**. ANSI é a sigla para *American National Standards Institute*, o órgão que na década de 1970 padronizou uma forma universal de enviar instruções de formatação para terminais de texto. Em vez de cada fabricante de terminal inventar seus próprios códigos, o padrão ANSI definiu sequências de caracteres que qualquer terminal compatível saberia interpretar, desde cores e negrito até posicionamento do cursor. Hoje, praticamente todos os terminais modernos, incluindo o do Linux, suportam esse padrão. Se quiser se aprofundar na especificação completa, a referência mais consultada está na [Wikipedia - ANSI escape code](#).

Para o nosso uso, é interessante, que essas sequências permitem colorir o texto diretamente no terminal. Cores podem parecer um detalhe estético, mas em scripts de monitoramento elas carregam informação real. Um processo operando normalmente aparece em verde, um alerta em amarelo e uma falha crítica em vermelho. O operador identifica o problema antes mesmo de terminar de ler a mensagem.

A sequência segue este formato fixo:

```
\033[CODIGOm texto que será colorido \033[0m
```

Parte	O que faz
<code>\033</code>	Caractere de escape que avisa o terminal que vem uma instrução de formatação
<code>[CODIGOm</code>	O código da cor ou estilo a aplicar
<code>\033[0m</code>	Reseta toda a formatação de volta ao padrão do terminal

O `\033[0m` no final é obrigatório. Sem ele, a cor vaza para todo o texto que aparecer depois no terminal. Os códigos de cor mais usados são:

Cor	Normal	Exemplo normal	Negrito	Exemplo negrito
Preto	30	<code>\033[30m texto \033[0m</code>	1;30	<code>\033[1;30m texto \033[0m</code>
Vermelho	31	<code>\033[31m texto \033[0m</code>	1;31	<code>\033[1;31m texto \033[0m</code>
Verde	32	<code>\033[32m texto \033[0m</code>	1;32	<code>\033[1;32m texto \033[0m</code>
Amarelo	33	<code>\033[33m texto \033[0m</code>	1;33	<code>\033[1;33m texto \033[0m</code>
Azul	34	<code>\033[34m texto \033[0m</code>	1;34	<code>\033[1;34m texto \033[0m</code>
Roxo	35	<code>\033[35m texto \033[0m</code>	1;35	<code>\033[1;35m texto \033[0m</code>
Ciano	36	<code>\033[36m texto \033[0m</code>	1;36	<code>\033[1;36m texto \033[0m</code>
Branco	37	<code>\033[37m texto \033[0m</code>	1;37	<code>\033[1;37m texto \033[0m</code>

Para ver na prática, crie o arquivo `scripts/formatos.sh`:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ nano scripts/formatos.sh
```

Adicione as linhas de cor ao final do arquivo:

```
#!/bin/bash

echo -e "Status do sistema:\nTodos os robôs operando"
echo -e "Robo\tBateria\tStatus\tObservações"
echo -e "robo A\t87%\t\t\033[32mOnline\033[0m\t\t\033[1;32mCarregado\033[0m"
echo -e "robo B\t43%\t\t\033[32mOnline\033[0m\t\t\033[1;33mNormal\033[0m"
echo -e "robo C\t43%\t\t\033[32mOnline\033[0m\t\t\033[1;33mNormal\033[0m"
echo -e "robo D\t13%\t\t\033[32mOnline\033[0m\t\t\033[1;31mPrecisa
carregar\033[0m"
```

Salve com **Ctrl+O** e saia com **Ctrl+X**. Mude a permissão do arquivo para um executável. Ao executar, temos o seguinte resultado:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/formatos.sh
Status do sistema:
Todos os robôs operando
Robo      Bateria Status  Observações
robo A    87% Online  Carregar
robo B    43% Online  Normal
robo C    43% Online  Normal
robo D    13% Online  Precisa carregar
```

1.2 Variáveis e o operador \$

Antes de criar nossas próprias variáveis, precisamos entender o operador que torna tudo isso possível: o `$`. No Bash, uma variável funciona exatamente como na matemática, você dá um nome a um espaço de memória e armazena um valor nele, mas escrever o nome sozinho não faz nada, é só um rótulo. O `$` é o operador que diz ao shell para ir até aquela variável e trazer o valor guardado dentro dela. Pense assim: `HOME` é o rótulo da caixa, `$HOME` é o conteúdo que está dentro dela. O sistema já mantém diversas variáveis preenchidas automaticamente desde o momento em que você abre o terminal, as mais úteis no dia a dia são:

Variável	O que guarda
\$HOME	Caminho do diretório pessoal do usuário
\$USER	Nome do usuário logado
\$PWD	Diretório atual onde você está
\$PATH	Lista de diretórios onde o shell busca programas
\$SHELL	Caminho do interpretador em uso

Você pode consultar qualquer uma delas diretamente no terminal com o **echo**:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo $HOME
/home/thallys

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo $PWD
/home/thallys/Documentos/Sonho/W2G

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo $USER
thallys
```

Quando o shell encontra o `$` dentro de um texto, ele substitui a variável pelo seu valor antes de executar qualquer coisa. É esse comportamento que permite misturar texto fixo com valores dinâmicos no mesmo `echo`, e é o mecanismo central por trás de praticamente tudo no Bash.

Até aqui, todas as variáveis que vimos eram preenchidas pelo próprio sistema. Mas scripts úteis de verdade precisam receber informações de fora, do usuário que os executa. Imagine um script que verifica o status de um robô específico: ele precisa saber qual robô verificar, e essa informação precisa chegar de algum lugar. É exatamente para isso que existem os argumentos posicionais, a forma mais direta de passar dados para um script no momento em que ele é chamado.

1.2.1 Argumentos Posicionais

Quando um script é executado, o shell lê tudo que foi digitado na linha de comando e distribui automaticamente cada valor em variáveis internas chamadas de **parâmetros posicionais**. O shell realiza essa separação usando os espaços em branco como divisores, e o preenchimento das variáveis acontece antes mesmo de processar a primeira linha do código. Esse mecanismo é o que permite que dados externos sejam passados para dentro do script no momento da execução.

```
./<script_bash> <$1> <$2> ...
```

Para acessar esses valores internamente, o Bash disponibiliza um conjunto de variáveis especiais que capturam desde argumentos individuais até informações sobre a própria execução:

Variável	Conteúdo
<code>\$0</code>	O nome do script ou comando sendo executado
<code>\$1 ... \$9</code>	Os argumentos posicionais individuais, indexados pela ordem em que foram passados
<code>\$*</code>	Todos os argumentos passados reunidos em uma única string
<code>@</code>	Todos os argumentos passados tratados como uma lista de elementos independentes
<code>#</code>	A quantidade total de argumentos recebidos, excluindo o nome do próprio script
<code>?</code>	O código de saída do último comando executado, onde 0 significa sucesso

Aplicando em um cenário concreto: Imagine que queremos criar um script chamado `saudacao.sh` que recebe o seu nome e sua idade. Ao executá-lo, por exemplo, `./saudacao.sh Thallys 25`, o shell separa

cada palavra e preenche as variáveis na ordem — `$1` recebe `Thallys` e `$2` recebe `25`. Quando o código começa a rodar, os valores já estão disponíveis, prontos para serem usados em qualquer parte do script.

```
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$ nano
scripts/saudacao.sh
```

```
#!/bin/bash

echo "=== Informações do Script ==="
echo "Script:          $0"
echo "Usuário:          $USER"
echo "Diretório Atual:  $PWD"
echo " "
echo "=== Argumentos Recebidos ==="
echo "Nome:             $1"
echo "Idade:             $2"
echo "Todos:             $"
echo "Quantidade:        $#"
```

Salve com `Ctrl+O` e saia com `Ctrl+X`. Conceda permissão de execução e rode passando os dois argumentos:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/saudacao.sh
Thallys 25
=== Informações do Script ===
Script:          ./scripts/saudacao.sh
Usuário:         thallys
Diretório atual: /home/thallys/Documentos/Sonho/W2G

=== Argumentos Recebidos ===
Nome:           Thallys
Idade:          25
Todos:          Thallys 25
Quantidade:     2
```

Observe que o script mistura os três tipos de variável sem nenhuma diferença de sintaxe. O `$USER` e o `$PWD` são variáveis de ambiente mantidas pelo sistema, o `$0` é uma variável especial do Bash, e o `$1` e o `$2` são variáveis posicionais preenchidas na hora da execução. Para o `echo`, todos são simplesmente valores acessados com o `$`, independentemente de onde vieram.

Entre todas essas variáveis, vale destacar a diferença entre `$*` e `$@`: o `$*` trata todos os argumentos como uma única palavra, enquanto o `$@` os preserva como elementos individuais. Na prática, `$@` é o mais seguro de usar quando você precisa repassar argumentos para outros comandos dentro do script, pois garante que argumentos com espaços não sejam quebrados inadvertidamente.

1.2.2 Criando suas Próprias Variáveis

Até agora, todas as variáveis que vimos eram preenchidas automaticamente: pelo sistema operacional (`$HOME`, `$USER`) ou pelo próprio Bash (`$1`, `$?`). Mas você também pode criar as suas próprias. Antes de ver como, vale entender que o Bash é **fracamente tipado**: diferente de linguagens como C, C++ ou Python onde você declara explicitamente `int`, `float` ou `char`, aqui todas as variáveis são internamente strings. O Bash interpreta o tipo pelo contexto em que a variável é usada:

Tipo	Exemplo	Notas
String	<code>nome="Thallys"</code>	Tipo padrão de tudo.
Inteiro	<code>idade=25</code>	Aritmética com <code>\$(())</code> .
Float	Não existe nativamente	Use <code>bc</code> para cálculos com decimais, veremos logo abaixo.
Booleano	Não existe nativamente	Usa-se <code>0</code> (verdadeiro) e <code>1</code> (falso) como código de saída.

1.2.2.1 String

String é o tipo padrão do Bash. Qualquer valor atribuído a uma variável é tratado como texto por padrão, mesmo que seja um número. Crie o arquivo `scripts/strings.sh` com o seguinte conteúdo:

```
#!/bin/bash

nome="Thallys"
cidade="São Carlos"
mensagem="Robô iniciado com sucesso"

echo "Olá, $nome!"
echo "Cidade: $cidade"
echo "Mensagem: $mensagem"
```

Ao executar, o `$` antes do nome da variável é substituído pelo valor armazenado nela:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/strings.sh
Olá, Thallys!
Cidade: São Carlos
Mensagem: Robô iniciado com sucesso
```

Aspas: uma diferença que muda tudo

O comportamento do `$` muda dependendo do tipo de aspas que você usa. Abra o arquivo `scripts/strings.sh` e adicione as linhas abaixo ao final:

```
echo "Olá, $nome" # aspas duplas: o $ é expandido
echo 'Olá, $nome' # aspas simples: tudo é literal
echo Olá, $nome  # sem aspas: funciona, mas perigoso com espaços
```


Ao executar novamente, repare como cada caso se comporta de forma diferente:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/strings.sh
Olá, Thallys!
Cidade: São Carlos
Mensagem: Robô iniciado com sucesso
Olá, Thallys
Olá, $nome
Olá, Thallys
```

Com aspas duplas o `$` é expandido e o valor da variável aparece na saída. Com aspas simples tudo é tratado como texto literal, então `$nome` aparece como está escrito, sem substituição. Sem aspas o comportamento é igual ao das aspas duplas, mas se o valor da variável contiver espaços o Bash o quebrará em partes separadas, o que pode causar erros inesperados. Por isso, aspas duplas são sempre a escolha mais segura.

Tipo	Comportamento
<code>"\$var"</code>	Interpreta e expande variáveis. Use sempre que houver <code>\$</code> .
<code>'\$var'</code>	Tudo é texto literal, nenhuma substituição acontece.

Sem espaços ao redor

Se colocarmos um espaço no `=` em `nome="Thallys"`, tanto antes, como depois, o shell interpretará `nome` como um comando, e não como uma variável. Abra o `scripts/strings.sh` e altere a linha `nome="Thallys"` para `nome = "Thallys"` e execute novamente:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/strings.sh
./scripts/strings.sh: linha 3: Thallys: comando não encontrado
Olá, !
Cidade: São Carlos
Mensagem: Robô iniciado com sucesso
Olá,
Olá, $nome
Olá,
```

Repare que o Bash tentou executar `Thallys` como um comando e falhou. Como a variável `nome` nunca foi definida corretamente, todas as linhas que dependiam dela aparecem vazias, exceto a com aspas simples, que imprime o texto literal `$nome` independente do valor da variável. Corrija o arquivo voltando para `nome="Thallys"` antes de continuar.

1.2.2.2 Inteiro

Inteiros são números sem parte decimal. No Bash, eles são declarados da mesma forma que strings: basta atribuir um valor numérico à variável, sem aspas. Crie o arquivo `scripts/inteiros.sh`:

```
#!/bin/bash

idade=25
ano=2025
quantidade=10

echo "Idade: $idade"
echo "Ano: $ano"
echo "Quantidade: $quantidade"
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/inteiros.sh
Idade: 25
Ano: 2025
Quantidade: 10
```

Repare que a declaração é idêntica à de uma string. A diferença aparece quando você precisa fazer cálculos com esses valores, o que veremos na seção 1.3 dedicada aos operadores.

1.2.2.3 Float

O Bash não suporta ponto flutuante nativamente. Qualquer divisão feita diretamente no Bash trunca o resultado para inteiro: `10 / 3` retorna `3`, não `3.33`. Para cálculos com decimais, utilizamos o `bc` (*Basic Calculator*), uma calculadora de precisão que já vem instalada na maioria dos sistemas Linux.

O `bc` funciona através de um pipe: escrevemos a expressão matemática como uma string e enviamos para ele processar. Variáveis do Bash podem ser usadas normalmente dentro da string, pois o `$` é expandido antes de o pipe ser executado. Crie o arquivo `scripts/float.sh`:

```
#!/bin/bash

valor=10

echo "$valor / 3" | bc          # 3      - sem scale, trunca
echo "scale=2; $valor / 3" | bc # 3.33  - duas casas decimais
echo "scale=4; $valor / 3" | bc # 3.1428 - quatro casas decimais
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/float.sh
3
3.33
3.3333
```

O `scale` e a expressão são separados por `;` dentro da mesma string porque o `bc` interpreta isso como duas instruções em sequência: primeiro define a precisão, depois executa o cálculo. Os espaços ao redor do `/` não são obrigatórios, `$valor/3` funcionaria igual, mas são uma boa prática de legibilidade.

1.2.2.4 Booleano

O Bash não possui um tipo booleano nativo. Em vez disso, utiliza o **código de saída** dos comandos: `0` significa sucesso (verdadeiro) e `1` significa falha (falso).

Todo comando executado no Bash termina com um código de saída, que o shell automaticamente armazena na variável especial `$?`. Esse valor é sobrescrito a cada novo comando, então para preservá-lo usamos uma variável própria: `status=$?` captura o valor de `$?` no momento em que a linha é executada e guarda em `status` antes que o próximo comando o sobrescreva. Crie o arquivo `scripts/booleano.sh`:

```
#!/bin/bash

true                                # executa – código de saída vai para
$?                                  # captura $? antes que o próximo
status=$?                           comando o sobrescreva
echo "true retornou: $status"        # true retornou: 0 – verdadeiro

false                                # executa – código de saída vai para
$?                                  # captura $?
status=$?                           # false retornou: 1 – falso
echo "false retornou: $status"

cd /pasta_inexistente                # executa – falha, código de saída vai
para $?                             # captura $?
status=$?                           # cd inválido retornou: 1 – falso
echo "cd inválido retornou: $status"
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/booleano.sh
true retornou: 0
false retornou: 1
./scripts/booleano.sh: linha 10: cd: /pasta_inexistente: Arquivo ou
diretório inexistente
cd inválido retornou: 1
```

Repare que o `cd` para uma pasta inexistente retornou `1`, mesmo gerando uma mensagem de erro na tela. O código de saída e a mensagem de erro são coisas separadas: um vai para `$?`, o outro vai para o terminal. Veremos como usar isso com operadores de comparação e condicionais nas próximas seções, onde o código de saída é a base de todo `if/else`.

1.2.2.5 `readonly` e `unset`

Até agora, todas as variáveis que criamos podiam ser modificadas ou sobrescritas a qualquer momento. O Bash oferece dois comandos para controlar o ciclo de vida de uma variável: o `readonly`, que congela o valor e impede qualquer modificação posterior, e o `unset`, que faz o oposto e remove a variável completamente da memória. Ambos funcionam com qualquer tipo. Crie o arquivo `scripts/readonly_unset.sh`:

```
#!/bin/bash

readonly VERSAO="1.0.0"
readonly MAX_ROBOS=10

robo_ativo="roboA"

echo "Versão: $VERSAO"
echo "Máximo de robôs: $MAX_ROBOS"
echo "Robô ativo: $robo_ativo"

VERSAO="2.0.0"
echo "Versão: $VERSAO"          # não muda – readonly bloqueou

unset robo_ativo
echo "Robô ativo: $robo_ativo"  # vazio – variável removida
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$
./scripts/readonly_unset.sh
Versão: 1.0.0
Máximo de robôs: 10
Robô ativo: roboA
./scripts/readonly_unset.sh: linha 13: VERSAO: variável somente leitura
Versão: 1.0.0
Robô ativo:
```

O `readonly` é útil para proteger valores que não devem mudar durante a execução do script, como configurações e constantes. O `unset` faz o oposto: remove a variável completamente, e não pode ser usado em variáveis declaradas com `readonly`.

1.3 Operadores

1.3.1 Substituição de Comando `$()`

Até agora, todas as variáveis que criamos receberam valores fixos escritos diretamente no código. A substituição de comando `$()` vai além disso: ela permite capturar a saída de qualquer comando e armazená-la em uma variável, em uma string ou até dentro de outro comando. O shell executa o que está dentro dos parênteses, captura o resultado e o substitui no lugar antes de continuar a execução.

Para ver na prática, crie o arquivo `scripts/substituicao.sh`:

```
#!/bin/bash

data_atual=$(date +"%Y-%m-%d %H:%M:%S")
diretorio=$(pwd)
total_arquivos=$(ls | wc -l)
usuario_logado=$(whoami)
arvore_diretorio=$(tree)

echo "Data: $data_atual"
echo "Diretório: $diretorio"
echo "Arquivos neste diretório: $total_arquivos"
echo "Usuário: $usuario_logado"
echo "Árvore de Arquivos: $arvore_diretorio"
```

O resultado é:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$
./scripts/substituicao.sh
Data: 2026-04-20 19:04:58
Diretório: /home/thallys/Documentos/Sonho/W2G
Arquivos neste diretório: 18
Usuário: thallys
Árvore de Arquivos: .
├── configs
│   ├── roboA.cfg
│   ├── roboB.cfg
│   ├── roboC.cfg
│   └── roboZ.cfg
├── logs
│   ├── acessos.log
│   ├── erro_critico.log
│   └── operacoes.log
├── scripts
│   ├── booleano.sh
│   ├── float.sh
│   ├── formatos.sh
│   ├── substituicao.sh
│   └── variaveis.sh
└── testes
    ├── teste1.py
    └── teste2.py
(...)

5 directories, 44 files
```

Repare o que aconteceu: cinco comandos diferentes foram executados e seus resultados armazenados em variáveis antes de qualquer `echo` ser chamado. O `date` formatou a data no padrão `ano-mês-dia hora:minuto:segundo` graças ao argumento `+"%Y-%m-%d %H:%M:%S"`, o `pwd` capturou o diretório atual, o `ls | wc -l` contou os arquivos, o `whoami` identificou o usuário e o `tree` mapeou toda a estrutura do

projeto em uma única string. Tudo isso aconteceu silenciosamente, antes de qualquer saída aparecer no terminal.

O `$()` não se limita a variáveis. Ele pode ser usado diretamente dentro de strings e até aninhado dentro de outros comandos. Veremos isso na prática a seguir, começando pelos operadores aritméticos, que também possuem sua própria sintaxe de substituição: o `$(( ))`.

1.3.2 Operadores Aritméticos `$(( ))`

O `$(( ))` é o operador de aritmética inteira do Bash. Tudo que estiver dentro dos parênteses duplos é interpretado como uma expressão matemática, e o resultado substitui o `$(( ))` no lugar onde foi usado, da mesma forma que o `$()` faz com comandos.

Crie o arquivo `scripts/aritmetica.sh`:

```
#!/bin/bash

a=15
b=4

echo "Soma:           $((a + b))"      # 19
echo "Subtração:      $((a - b))"      # 11
echo "Multiplicação:  $((a * b))"      # 60
echo "Divisão:        $((a / b))"      # 3 – divisão inteira, sem decimal
echo "Módulo:         $((a % b))"      # 3 – resto da divisão
echo "Potência:       $((a ** 2))"     # 225

((a++))
echo "Incremento:     $a"              # 16

((a--))
echo "Decremento:     $a"              # 15

((a += 10))
echo "Soma direta:    $a"              # 25

((a -= 3))
echo "Subtração direta: $a"            # 22

((a *= 2))
echo "Multiplicação direta: $a"        # 44

((a /= 4))
echo "Divisão direta:  $a"              # 11
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/aritmetica.sh
Soma:           19
Subtração:      11
```

```

Multiplicação:      60
Divisão:            3
Módulo:             3
Potência:           225
Incremento:         16
Decremento:         15
Soma direta:        25
Subtração direta:   22
Multiplicação direta: 44
Divisão direta:     11

```

O incremento `((a++))` e o decremento `((a--))` são formas curtas de somar ou subtrair 1 da variável. A mesma sintaxe funciona para qualquer operação e qualquer valor, basta trocar o operador: `+=`, `-=`, `*=` e `/=` modificam a variável diretamente sem precisar do `$`, pois não estamos capturando um valor para uso imediato, apenas atualizando a variável na memória.

1.3.3 Operadores de Comparação []

Os operadores de comparação são usados dentro de `[ ]` para comparar valores e retornar um código de saída: `0` se a condição for verdadeira e `1` se for falsa. O Bash divide esses operadores em categorias, cada uma com sua própria sintaxe.

- **Números**

Para comparar números inteiros, o Bash usa flags de letras em vez de símbolos como `>` ou `==`, pois esses já têm outros significados no shell. Floats não são suportados diretamente dentro de `[ ]`, para isso seria necessário usar o `bc`:

Operador	Significado	Exemplo
<code>-eq</code>	igual (equal)	<code>[\$a -eq \$b]</code>
<code>-ne</code>	diferente (not equal)	<code>[\$a -ne \$b]</code>
<code>-gt</code>	maior (greater than)	<code>[\$a -gt \$b]</code>
<code>-ge</code>	maior ou igual	<code>[\$a -ge \$b]</code>
<code>-lt</code>	menor (less than)	<code>[\$a -lt \$b]</code>
<code>-le</code>	menor ou igual	<code>[\$a -le \$b]</code>

Crie o arquivo `scripts/comparacao_numeros.sh`:

```

#!/bin/bash

bateria=75
limite_critico=20
limite_atencao=50

[ $bateria -gt $limite_atencao ]
echo "Bateria acima de $limite_atencao%: $?" # 0 – verdadeiro

```

```
[ $bateria -lt $limite_critico ]
echo "Bateria abaixo de $limite_critico%: $?"      # 1 – falso

[ $bateria -eq 75 ]
echo "Bateria exatamente em 75%: $?"              # 0 – verdadeiro
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$
./scripts/comparacao_numeros.sh
Bateria acima de 50%: 0
Bateria abaixo de 20%: 1
Bateria exatamente em 75%: 0
```

Strings

Para comparar strings, usamos símbolos. As aspas duplas ao redor das variáveis são importantes: sem elas, uma variável vazia quebraria o comando pois o Bash tentaria interpretar o espaço vazio como parte da sintaxe do `[ ]`:

Operador	Significado	Exemplo
<code>=</code>	iguais	<code>["\$a" = "\$b"]</code>
<code>!=</code>	diferentes	<code>["\$a" != "\$b"]</code>
<code>-z</code>	vazia	<code>[-z "\$a"]</code>
<code>-n</code>	não vazia	<code>[-n "\$a"]</code>

Crie o arquivo `scripts/comparacao_strings.sh`:

```
#!/bin/bash

status_robo="online"
status_esperado="online"
mensagem_erro=""

[ "$status_robo" = "$status_esperado" ]
echo "Status correto: $?"      # 0 – verdadeiro, strings são iguais

[ "$status_robo" != "offline" ]
echo "Robô não está offline: $?"      # 0 – verdadeiro, strings são diferentes

[ -z "$mensagem_erro" ]
echo "Sem mensagem de erro: $?"      # 0 – verdadeiro, string está vazia

[ -n "$status_robo" ]
```



```
echo "Status preenchido: $?"          # 0 – verdadeiro, string não está vazia
```

Ao executar:

```
(base)                                thallys@thallys-note:~/Documentos/Sonho/W2G$  
./scripts/comparacao_strings.sh  
Status correto: 0  
Robô não está offline: 0  
Sem mensagem de erro: 0  
Status preenchido: 0
```

Arquivos

O Bash possui a capacidade de analisar propriedades de arquivos e diretórios, como verificar a existência e as permissões de acesso. Para isso, utilizamos operadores específicos que são muito úteis em scripts: antes de tentar ler ou executar um arquivo, por exemplo, podemos verificar se ele existe e se temos permissão para acessá-lo, evitando erros inesperados durante a execução:

Operador	Significado	Exemplo
-f	é um arquivo regular	[-f "log.txt"]
-d	é um diretório	[-d "/tmp"]
-e	existe	[-e "algo"]
-r / -w / -x	legível / gravável / executável	[-x "script.sh"]
!	negação	[! -f "log.txt"]

Crie o arquivo `scripts/comparacao_arquivos.sh`:

```
#!/bin/bash  
  
log="./robot_logs/roboA_2025-03-19.log"  
diretorio="./robot_logs"  
script="./scripts/monitor_robos.sh"  
  
[ -f "$log" ]  
echo "Log existe: $?"          # 0 – verdadeiro, arquivo existe  
  
[ -d "$diretorio" ]  
echo "Diretório existe: $?"    # 0 – verdadeiro, diretório existe  
  
[ -r "$log" ]  
echo "Log é legível: $?"      # 0 – verdadeiro, temos permissão de leitura  
leitura  
  
[ -x "$script" ]
```

```

echo "Script é executável: $?"          # 0 ou 1 – depende da permissão do
arquivo

[ ! -f "arquivo_inexistente.log" ]
echo "Arquivo não existe: $?"          # 0 – verdadeiro, arquivo realmente
não existe

```

Ao executar:

```

(base)                                thallys@thallys-note:~/Documentos/Sonho/W2G$
./scripts/comparacao_arquivos.sh
Log existe: 1
Diretório existe: 1
Log é legível: 1
Script é executável: 1
Arquivo não existe: 0

```

Operadores Lógicos

Os operadores lógicos combinam duas ou mais condições em uma única expressão, evitando a necessidade de verificar cada condição separadamente:

Operador	Significado	Exemplo
-a ou &&	E (AND) — verdadeiro se ambas as condições forem verdadeiras	[\$a -gt 0 -a \$a -lt 100]
-o ou	OU (OR) — verdadeiro se pelo menos uma condição for verdadeira	[\$a -eq 0 -o \$a -eq 1]

Crie o arquivo `scripts/comparacao_logica.sh`:

```

#!/bin/bash

bateria=75
status="online"
log="./robot_logs/roboA_2025-03-19.log"

[ $bateria -gt 20 -a "$status" = "online" ]
echo "Bateria ok E robô online: $?"          # 0 – verdadeiro, ambas
verdadeiras

[ $bateria -lt 20 -o "$status" = "online" ]
echo "Bateria crítica OU robô online: $?"    # 0 – verdadeiro, pelo menos
uma verdadeira

[ $bateria -lt 20 -a -f "$log" ]

```

```
echo "Bateria crítica E log existe: $?" # 1 – falso, bateria não está  
abaixo de 20 e o arquivo não existe
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$  
./scripts/comparacao_logica.sh  
Bateria ok E robô online: 0  
Bateria crítica OU robô online: 0  
Bateria crítica E log existe: 1
```

Repare no terceiro exemplo: o resultado foi **1** porque o **-a** exige que ambas as condições sejam verdadeiras, e nenhuma delas foi: a bateria não está abaixo de 20 e o arquivo de log ainda não existe no diretório.

Decisões rápidas com **&&** e **||**

Os mesmos operadores **&&** e **||** podem ser usados fora dos colchetes para controlar o fluxo de execução diretamente. Enquanto dentro do **[]** eles combinam condições, fora deles decidem o que executar com base no código de saída do comando anterior: o **&&** executa o comando da direita somente se o da esquerda tiver sucesso, e o **||** executa somente se falhar:

```
mkdir backups && echo "Diretório criado com sucesso"  
mkdir backups || echo "Erro: diretório já existe"
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ mkdir backups && echo  
"Diretório criado com sucesso"  
Diretório criado com sucesso  
  
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ mkdir backups || echo  
"Erro: diretório já existe"  
mkdir: não foi possível criar o diretório 'backups': Arquivo existe  
Erro: diretório já existe
```

No primeiro comando, o **mkdir** teve sucesso e o **&&** permitiu que o **echo** fosse executado. No segundo, o **mkdir** falhou porque o diretório já existia, então o **||** disparou a mensagem de erro.

1.3.4 Operador de Chaves **\${}**

O **\${}** é uma forma mais explícita e segura de acessar variáveis no Bash. Na maioria dos casos, **\$VARIÁVEL** e **\${VARIÁVEL}** se comportam da mesma forma, mas as chaves se tornam obrigatórias quando o nome da variável está colado a outros caracteres e o Bash precisa saber exatamente onde o nome termina.

O exemplo abaixo retoma o script de cores da seção 1.1.3, mas desta vez as cores são definidas como variáveis. Sem as chaves, o Bash não conseguiria distinguir onde termina o nome da variável e onde começa o código ANSI ao redor. Salve como `scripts/chaves.sh`:

```
#!/bin/bash

VERDE="\033[32m"
AMARELO="\033[33m"
VERMELHO="\033[31m"
RESET="\033[0m"

echo -e "Status do sistema:\nTodos os robôs operando"
echo -e "Robo\tBateria\tStatus\tObservações"
echo -e "robo A\t87%\t${VERDE}Online${RESET}\t${VERDE}Normal${RESET}"
echo -e "robo B\t43%\t${VERDE}Online${RESET}\t${AMARELO}Atenção${RESET}"
echo -e "robo C\t43%\t${VERDE}Online${RESET}\t${AMARELO}Atenção${RESET}"
echo -e "robo D\t13%\t${VERDE}Online${RESET}\t${VERMELHO}Precisa carregar${RESET}"
```

Ao executar, o resultado será a mesma tabela colorida da seção 1.1.3. Repare na diferença em relação ao exemplo anterior, onde as cores eram escritas diretamente em cada linha:

```
# Antes – código exposto e difícil de manter
echo -e "robo A\t87%\t\033[32mOnline\033[0m\t\033[1;32mNormal\033[0m"

# Depois – limpo e imediatamente legível
echo -e "robo A\t87%\t${VERDE}Online${RESET}\t${VERDE}Normal${RESET}"
```

Com variáveis, o código comunica a intenção: `${VERDE}` é imediatamente legível, enquanto `\033[32m` exige que o programador consulte a tabela toda vez. Além disso, se precisar trocar uma cor, basta alterar a variável no topo do script em vez de procurar e substituir o código ANSI em cada linha.

O `${}` não se limita a cores. Ele é obrigatório em qualquer situação onde o `$` sozinho causaria ambiguidade. Veja o problema sem as chaves e como elas resolvem, crie o arquivo `scripts/main.sh`:

```
#!/bin/bash
arquivo="robo"

echo "$arquivocfg" # vazio – Bash procurou por "arquivocfg"
echo "./configs/${arquivo}.cfg" # ./configs/robo.cfg – variável isolada corretamente
echo "${arquivo}_2025" # robo_2025 – mesmo comportamento com underscore

robo="roboA"
```

```
echo "./configs/${robo}.cfg"          # ./configs/roboA.cfg – constrói o
caminho
cat "./configs/${robo}.cfg"           # lê o conteúdo do arquivo roboA.cfg
ls "${robo}_2025-03-19.log"           # arquivo não existe – erro esperado
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/main.sh

./configs/robo.cfg
robo_2025
./configs/roboA.cfg
Olá, eu sou o ROBO A!
ls: não foi possível acessar 'roboA_2025-03-19.log': Arquivo ou diretório
inexistente
```

Sempre que o nome da variável estiver colado a outros caracteres, use `${}` para delimitar onde o nome começa e termina. Como vimos no exemplo, sem as chaves o Bash tentou resolver `$arquivocfg` como uma única variável, retornando vazio. Com as chaves, `${arquivo}.cfg` deixa claro que `arquivo` é a variável e `.cfg` é texto literal.

1.4 Condicionais (`if/else`)

A estrutura condicional no Bash segue uma sintaxe própria que difere bastante de outras linguagens. O bloco começa com `if` e termina com `fi`, que é o `if` invertido. O `then` marca o início do bloco que será executado se a condição for verdadeira, e o `elif` permite encadear condições adicionais antes do `else` final. As condições são escritas dentro de `[ ]`, com **espaços obrigatórios** após o `[` e antes do `]`: sem esses espaços, o Bash retorna erro de sintaxe.

```
if [ condição ]; then
    comandos
elif [ condição ]; then
    comandos
else
    comandos
fi
```

O `if` avalia o código de saída da condição: `0` executa o bloco, qualquer outro valor pula para o `elif` ou `else`. Para exemplificar, crie o arquivo `scripts/condicional.sh`:

```
#!/bin/bash

bateria=43

if [ "${bateria}" -ge 80 ]; then
    echo "Bateria OK"
```

```
elif [ "${bateria}" -ge 20 ]; then
    echo "Bateria em atenção"
else
    echo "Bateria crítica"
fi
```

Ao executar, o Bash avalia cada condição de cima para baixo e executa o primeiro bloco cuja condição for verdadeira, ignorando todos os outros:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$
./scripts/condicional.sh
Bateria em atenção
```

Um uso mais prático é validar o ambiente antes de executar qualquer operação. Antes de criar o script, crie o arquivo `tmp.txt` com o seguinte conteúdo:

```
roboA.cfg
roboB.cfg
roboC.cfg
roboZ.cfg
```

Agora crie o arquivo `scripts/validacao.sh`. O script verifica se um argumento foi passado e se o arquivo existe antes de processá-lo:

```
#!/bin/bash

if [ $# -eq 0 ]; then
    echo "Erro: nenhum arquivo informado."
    echo "Uso: $0 <arquivo>"
    exit 1
fi

if [ ! -f "$1" ]; then
    echo "Erro: arquivo '$1' não encontrado."
    exit 1
fi

echo "Processando $1..."
echo "Total de linhas: $(wc -l < $1)"
```

Ao executar sem argumento e depois passando o `tmp.txt`:

```
# Sem argumento
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/validacao.sh
Erro: nenhum arquivo informado.
```

```
Uso: ./scripts/validacao.sh <arquivo>
```

```
# Com arquivo válido
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/validacao.sh
tmp.txt
Processando tmp.txt...
Total de linhas: 4
```

O `exit 1` encerra o script imediatamente ao encontrar um problema, evitando que erros em cascata se propaguem. Repare que o script só chega à linha de processamento se todas as validações anteriores passarem: primeiro verifica se recebeu um argumento, depois se o arquivo existe. Essa é a estrutura base de qualquer script robusto.

1.5 for

O `for` itera sobre uma lista de valores, executando o bloco entre `do` e `done` para cada item. A estrutura básica é:

```
for variavel in lista; do
    comandos
done
```

A cada iteração, a `variavel` recebe o próximo valor da lista e os comandos dentro do bloco são executados. Quando a lista acaba, o loop termina. A lista pode ser composta de palavras literais, uma sequência numérica, o conteúdo de uma variável ou a saída de um comando.

Crie o arquivo `scripts/loop_for.sh`:

```
#!/bin/bash

# Sobre palavras literais
for robo in roboA roboB roboC; do
    echo "Verificando: $robo"
done

# Sobre uma sequência numérica
for i in {1..5}; do
    echo "Tentativa: $i"
done

# Sobre o conteúdo de um arquivo
for linha in $(cat tmp.txt); do
    echo "Encontrado: $linha"
done

# Estilo C (quando precisa de contadores)
for ((i=0; i<3; i++)); do
```

```
    echo "Índice: $i"
done
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/loop_for.sh
Verificando: roboA
Verificando: roboB
Verificando: roboC
Tentativa: 1
Tentativa: 2
Tentativa: 3
Tentativa: 4
Tentativa: 5
Encontrado: roboA.cfg
Encontrado: roboB.cfg
Encontrado: roboC.cfg
Encontrado: roboZ.cfg
Índice: 0
Índice: 1
Índice: 2
```

O primeiro bloco itera sobre palavras literais separadas por espaço. O segundo usa `{1..5}` para gerar automaticamente uma sequência de 1 a 5. O terceiro lê o `tmp.txt` usando `$(cat tmp.txt)`, capturando a saída do comando e iterando sobre cada linha. O quarto usa a sintaxe estilo C, útil quando você precisa de controle preciso sobre o contador, como definir um passo específico ou iterar de trás para frente.

1.6 while

O `while` repete um bloco de comandos enquanto uma condição for verdadeira. A estrutura básica é:

```
while [ condição ]; do
    comandos
done
```

A cada iteração, a condição é avaliada. Se o código de saída for `0`, o bloco é executado. Quando a condição falhar, o loop termina. Crie o arquivo `scripts/loop_while.sh`:

```
#!/bin/bash

contador=1
while [ ${contador} -le 5 ]; do
    echo "Tentativa: ${contador}"
    ((contador++))
done
```


Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/loop_while.sh
Tentativa: 1
Tentativa: 2
Tentativa: 3
Tentativa: 4
Tentativa: 5
```

O **while** é especialmente útil para ler arquivos linha por linha. Diferente do **for**, que divide o conteúdo por espaços, o **while** com **read** preserva cada linha inteira. Adicione ao final do arquivo:

```
while read -r robo; do
    echo "Robô encontrado: ${robo}"
done < tmp.txt
```

Ao executar novamente:

```
Tentativa: 1
Tentativa: 2
Tentativa: 3
Tentativa: 4
Tentativa: 5
Robô encontrado: roboA.cfg
Robô encontrado: roboB.cfg
Robô encontrado: roboC.cfg
Robô encontrado: roboZ.cfg
```

O **read -r** lê uma linha por vez do arquivo e armazena em **robo**. O **< tmp.txt** redireciona o arquivo para o **stdin** do **while**, alimentando o **read** linha por linha até o arquivo acabar.

1.7 **break** e **continue**

O **break** e o **continue** são comandos que alteram o fluxo de execução dentro de qualquer loop, seja **for** ou **while**. A diferença entre eles é simples: o **break** encerra o loop imediatamente, enquanto o **continue** pula para a próxima iteração sem executar o restante do bloco.

break — saindo do loop

```

for/while ...; do
    if [ condição ]; then
        break          # encerra o loop aqui
    fi
    comandos
done

```

continue — pulando uma iteração

```

for/while ...; do
    if [ condição ]; then
        continue      # volta para o início do loop
    fi
    comandos
done

```

Para ver os dois em ação, crie o arquivo `scripts/controle_fluxo.sh`:

```

#!/bin/bash

echo "=== Testando continue ==="
for i in {1..5}; do
    if [ ${i} -eq 3 ]; then
        continue      # pula o 3
    fi
    echo "Número: ${i}"
done

echo "=== Testando break ==="
contador=1
while true; do
    if [ ${contador} -eq 4 ]; then
        break          # encerra o loop no 4
    fi
    echo "Contagem: ${contador}"
    ((contador++))
done

```

Ao executar:

```

(base)                                     thallys@thallys-note:~/Documentos/Sonho/W2G$
./scripts/controle_fluxo.sh
=== Testando continue ===
Número: 1
Número: 2
Número: 4

```

```
Número: 5
=== Testando break ===
Contagem: 1
Contagem: 2
Contagem: 3
```

No primeiro bloco, o `continue` pulou o número 3 e o loop seguiu normalmente até o fim. No segundo, o `while true` criaria um loop infinito, mas o `break` encerrou a execução assim que o contador chegou em 4.

1.8 Arrays

Um array é uma variável que armazena múltiplos valores em uma única estrutura, acessíveis por índice. No Bash, os elementos são separados por **espaço** e envolvidos por parênteses, sem vírgulas:

```
array=("elemento1" "elemento2" "elemento3")
```

Os índices começam em 0, e o acesso a um elemento específico é feito com `${array[indice]}`. Para acessar todos os elementos, usamos `${array[@]}`. Crie o arquivo `scripts/arrays.sh`:

```
#!/bin/bash

robos=("roboA" "roboB" "roboC" "roboZ")

echo "Primeiro robô: ${robos[0]}"          # roboA
echo "Terceiro robô: ${robos[2]}"          # roboC
echo "Todos os robôs: ${robos[@]}"         # roboA roboB roboC roboZ
echo "Total de robôs: ${#robos[@]}"        # 4

robos+=("roboX")
echo "Após adicionar: ${robos[@]}"         # roboA roboB roboC roboZ roboX

unset robos[1]
echo "Após remover: ${robos[@]}"           # roboA roboC roboZ roboX
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/arrays.sh
Primeiro robô: roboA
Terceiro robô: roboC
Todos os robôs: roboA roboB roboC roboZ
Total de robôs: 4
Após adicionar: roboA roboB roboC roboZ roboX
Após remover: roboA roboC roboZ roboX
```

A forma mais natural de percorrer um array é com o `for`. As aspas duplas ao redor de `${robos[@]}` são obrigatórias para preservar elementos que contenham espaços: sem elas, cada espaço seria interpretado como separador de elementos. Adicione ao final do arquivo:

```
sensores=("LIDAR" "câmera frontal" "GPS" "giroscópio")

for sensor in "${sensores[@]"; do
    echo "Verificando sensor: ${sensor}"
done
```

Ao executar novamente:

```
Verificando sensor: LIDAR
Verificando sensor: câmera frontal
Verificando sensor: GPS
Verificando sensor: giroscópio
```

Repare que `câmera frontal` foi preservado como um único elemento graças às aspas. Sem elas, o Bash separaria em `câmera` e `frontal`, quebrando o array. Embora o `while` também possa ser usado para percorrer arrays, o `for` é a escolha natural aqui pois já conhecemos o número de elementos.

Agora um exemplo mais completo, combinando array com condicionais para simular uma verificação de frota. Crie o arquivo `scripts/frota.sh`:

```
#!/bin/bash

robos=("roboA" "roboB" "roboC" "roboZ")
baterias=(87 15 43 95)

for i in "${!robos[@]"; do
    robo="${robos[$i]}"
    bateria="${baterias[$i]}"

    if [ "${bateria}" -ge 80 ]; then
        echo "${robo}: Bateria OK (${bateria}%)"
    elif [ "${bateria}" -ge 20 ]; then
        echo "${robo}: Bateria em atenção (${bateria}%)"
    else
        echo "${robo}: Bateria crítica (${bateria}%)"
    fi
done
```

Ao executar:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/frota.sh
roboA: Bateria OK (87%)
roboB: Bateria crítica (15%)
roboC: Bateria em atenção (43%)
roboZ: Bateria OK (95%)
```

O `${!robos[@]}` retorna os índices do array em vez dos valores. Sem o `!`, o `for` iteraria sobre os nomes dos robôs diretamente. Com o `!`, ele itera sobre os números `0`, `1`, `2`, `3`, que são as posições do array. Isso permite usar o mesmo índice `i` para acessar dois arrays ao mesmo tempo: `${robos[$i]}` pega o robô na posição `i` e `${baterias[$i]}` pega a bateria na mesma posição. É como duas colunas de uma tabela percorridas em paralelo pela mesma linha.

1.9 Funções

Funções encapsulam blocos de código reutilizáveis que podem ser chamados várias vezes ao longo do script. No Bash, a estrutura básica é:

```
nome_da_funcao() {
    comandos
}
```

A função é definida antes de ser chamada, e a chamada é feita pelo nome sem parênteses, passando os argumentos separados por espaço. Dentro da função, os argumentos são acessados via `$1`, `$2`, etc. Esses não são variáveis que você declara, são parâmetros posicionais preenchidos automaticamente no momento da chamada: quando você escreve `status_robo "roboA" 87`, o Bash coloca `"roboA"` em `$1` e `87` em `$2` dentro da função, da mesma forma que acontece com os argumentos do script principal.

A palavra-chave `local` limita o escopo da variável à função. Sem ela, a variável vaza para o escopo global e pode sobrescrever variáveis de mesmo nome em outras partes do script.

Crie o arquivo `scripts/funcoes.sh`:

```
#!/bin/bash

status_robo() {
    local robo="$1"
    local bateria="$2"
    echo "Robô: ${robo} — Bateria: ${bateria}%"
}

status_robo "roboA" 87
status_robo "roboB" 15
status_robo "roboC" 43
```

Execute o script:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/funcões.sh
```

O resultado é:

```
Robô: roboA — Bateria: 87%
Robô: roboB — Bateria: 15%
Robô: roboC — Bateria: 43%
```

Repare que a mesma função foi chamada três vezes com argumentos diferentes, evitando repetição de código. A cada chamada, o Bash preencheu `$1` e `$2` com os valores passados naquele momento, de forma independente entre as chamadas.

Agora um exemplo mais completo. No Bash, `return` define um código de saída (0–255) e não um valor. Para retornar dados, usamos `echo` dentro da função e capturamos com `$()`. Adicione ao final do arquivo:

```
verificar_bateria() {
    local bateria="$1"

    if [ "${bateria}" -lt 20 ]; then
        echo "CRÍTICO"
        return 1
    elif [ "${bateria}" -lt 50 ]; then
        echo "ATENÇÃO"
        return 1
    fi

    echo "OK"
    return 0
}

robos=("roboA" "roboB" "roboC")
baterias=(87 15 43)

for i in "${!robos[@]}"; do
    resultado=$(verificar_bateria "${baterias[$i]}")
    if [ $? -eq 0 ]; then
        echo "${robos[$i]}: ${resultado}"
    else
        echo "${robos[$i]}: ALERTA — ${resultado}"
    fi
done
```

Execute o script novamente:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ./scripts/funcoes.sh
```

O resultado é:

```
Robô: roboA — Bateria: 87%
Robô: roboB — Bateria: 15%
Robô: roboC — Bateria: 43%
roboA: OK
roboB: ALERTA — CRÍTICO
roboC: ALERTA — ATENÇÃO
```

A função `verificar_bateria` usa dois mecanismos de retorno ao mesmo tempo, e entender a diferença entre eles é essencial.

O `return` define o código de saída da função, que funciona exatamente como o `exit` em um script: `return 0` significa sucesso e `return 1` significa falha. Esse valor vai automaticamente para `$?` assim que a função termina. O problema é que `return` só aceita números de 0 a 255, não texto. Para retornar um valor de texto como "CRÍTICO" ou "OK", usamos `echo` dentro da função e capturamos com `$()` na chamada.

Isso cria um fluxo de duas informações saindo da função ao mesmo tempo: o texto vai pelo stdout e é capturado por `resultado=$(verificar_bateria ...)`, enquanto o código numérico vai para `$?`. A linha seguinte lê `$?` imediatamente, antes que qualquer outro comando o sobrescreva, e decide o que imprimir.

Acompanhe o fluxo para o `roboB` com bateria 15:

A função recebe 15, entra no primeiro `if` porque `15 -lt 20` é verdadeiro, faz `echo "CRÍTICO"` que vai para o stdout e `return 1` que vai para `$?`. Do lado de fora, `resultado` captura "CRÍTICO" e `$?` vale 1. O `if [ $? -eq 0 ]` falha, então cai no `else` e imprime `roboB: ALERTA — CRÍTICO`.

Para o `roboA` com bateria 87, nenhuma condição é satisfeita, a função chega ao `echo "OK"` e `return 0`. Do lado de fora `resultado` vale "OK" e `$?` vale 0, então o `if` executa e imprime `roboA: OK`.

1.10 PATH: Como o Shell Encontra os Programas

Ao longo dos exemplos, você notou que para rodar um script no diretório atual precisamos usar `./meu_script.sh`. Mas para rodar o `ls`, basta digitar `ls`. Por que a diferença?

Quando você digita um comando, o shell não vasculha o disco inteiro procurando pelo programa. Em vez disso, ele consulta uma lista pré-definida de diretórios chamada **PATH** e procura apenas neles, na ordem. Execute `echo $PATH` no seu terminal para ver quais são os seus:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo $PATH
/usr/local/cuda-
12.4/bin:/home/thallys/miniconda3/bin:/home/thallys/.local/share/zinit/pola
```

```
ris/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin
```

Cada diretório é separado por `:` e o shell os percorre da esquerda para a direita até encontrar o programa. O diretório atual não está nessa lista por segurança, por isso precisamos do `./` para rodar scripts locais.

O comando `which` revela em qual diretório do PATH o shell encontrou o programa:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ which ls
/usr/bin/ls

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ which python3
/home/thallys/miniconda3/bin/python3

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ which scripts/funcoes.sh
scripts/funcoes.sh
scripts/funcoes.sh

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ which scripts/ola.sh
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$
```

Repare que o `python3` foi encontrado dentro do `miniconda3` e não em `/usr/bin`, pois o diretório do Miniconda aparece antes no PATH. O `scripts/funcoes.sh` retornou o caminho relativo porque o passamos explicitamente na chamada do `which`, não porque estava no PATH. Já o `scripts/ola.sh` retornou vazio porque o `which` não o encontrou em nenhum diretório do PATH e também não recebeu um caminho explícito para ele.

O `whereis` é similar ao `which`, mas também mostra onde está o manual do programa:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ whereis python3
python3: /usr/bin/python3 /usr/lib/python3
/home/thallys/miniconda3/bin/python3 /usr/share/man/man1/python3.1.gz
```

Antes de adicionar o diretório ao PATH, vale entender dois atalhos importantes do Linux. O `~/` é um atalho para o diretório pessoal do usuário, equivalente a `/home/thallys/`. Então `~/Documentos` é o mesmo que `/home/thallys/Documentos`. Já o `~/.` acessa arquivos ocultos dentro do diretório pessoal, que são arquivos cujo nome começa com ponto. O `~/bashrc` por exemplo é o arquivo `.bashrc` dentro de `/home/thallys/`. Esses dois atalhos aparecem com frequência em configurações e scripts, por isso é importante reconhecê-los.

Para que seus scripts fiquem acessíveis de qualquer lugar sem o `./`, basta adicionar o diretório onde eles estão ao PATH:


```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ export  
PATH="$HOME/Documentos/Sonho/W2G/scripts:$PATH"
```

A partir desse momento, qualquer script dentro da pasta `scripts` pode ser chamado diretamente pelo nome, de qualquer diretório do sistema:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ funcoes.sh  
Robô: roboA – Bateria: 87%  
Robô: roboB – Bateria: 15%  
Robô: roboC – Bateria: 43%  
roboA: OK  
roboB: ALERTA – CRÍTICO  
roboC: ALERTA – ATENÇÃO
```

Confirmando que o diretório foi adicionado ao início do PATH:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo $PATH  
/home/thallys/Documentos/Sonho/W2G/scripts:/usr/local/cuda-  
12.4/bin:/home/thallys/miniconda3/bin:...
```

Repare que o diretório do projeto aparece no início da lista, o que significa que o shell o consultará primeiro antes de qualquer outro. Essa mudança é temporária e dura apenas até fechar o terminal. Para torná-la permanente, adicionamos a linha do `export` ao `~/.bashrc`, que veremos na seção 1.12.

1.11 Aliases: Atalhos para Comandos

Se os scripts automatizam sequências complexas, os aliases encurtam comandos que digitamos com frequência. Um alias é um apelido que o shell substitui pelo comando completo antes de executar. Para criar um alias, usamos a seguinte sintaxe:

```
alias nome='comando completo'
```

Crie os aliases abaixo diretamente no terminal:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ alias ll='ls -lh'  
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ alias cls='clear'  
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ alias projeto='cd  
~/Documentos/Sonho/W2G && ls'
```

O `ll` substitui o `ls -lh`, exibindo os arquivos em formato de lista com tamanhos legíveis. O `cls` substitui o `clear`, limpando a tela. O `projeto` combina dois comandos com `&&`, navegando até o diretório do projeto e listando o conteúdo em seguida. Para testar o `projeto`, navegue para outro diretório e execute:

```
(base) thallys@thallys-note:~/Documentos/Sonho$ projeto
configs docs logs scripts testes
```

Independente de onde você estiver, o alias leva direto ao projeto. Para listar todos os aliases ativos e remover um:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ alias
ll='ls -lh'
cls=clear
projeto='cd ~/Documentos/Sonho/W2G && ls'

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ unalias cls
```

Aliases criados diretamente no terminal são temporários e desaparecem ao fechar a sessão. Para torná-los permanentes, adicionamos ao `~/ .bashrc`, que veremos na próxima seção.

1.12 Variáveis de Ambiente e o `~/ .bashrc`

Variáveis de ambiente são configurações globais que o sistema e os programas consultam para entender o contexto em que estão rodando, como qual é o shell em uso, qual é o usuário logado e quais diretórios estão no PATH. O comando `env` lista todas as variáveis de ambiente ativas no momento:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ env | head -n 8
SHELL=/usr/bin/zsh
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
LESS=-R
```

A diferença entre uma variável comum e uma variável de ambiente está na visibilidade. Uma variável criada com `VAR="valor"` existe apenas no shell atual e desaparece quando você abre um subprocesso. O `export` promove a variável para o ambiente, tornando-a visível para todos os processos filhos que forem criados a partir dali:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ PROJETO="W2G"
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ bash -c 'echo $PROJETO'
# vazio - o
subprocesso não a herdou

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ export PROJETO="W2G"
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ bash -c 'echo $PROJETO'
W2G
# agora o subprocesso
a enxerga
```

O `~/ .bashrc` é o arquivo onde centralizamos todas as personalizações permanentes do shell. O nome vem de **Bash** mais **rc**, que é uma abreviação de *run commands*, ou seja, comandos que são executados automaticamente. O ponto no início do nome o torna um arquivo oculto, seguindo a convenção do Linux para arquivos de configuração. Toda vez que um novo terminal é aberto, o Bash lê e executa esse arquivo antes de qualquer coisa, o que significa que tudo que você colocar ali estará disponível em todas as sessões:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ nano ~/.bashrc

# Adicione ao final:
# ===== VARIÁVEIS DE AMBIENTE =====
export EDITOR="nano"
export ROBO_DIR="$HOME/Documentos/Sonho/W2G"

# ===== PATH =====
export PATH="$HOME/Documentos/Sonho/W2G/scripts:$PATH"

# ===== ALIASES =====
alias ll='ls -lh'
alias cls='clear'
alias projeto='cd $ROBO_DIR && ls'
```

Após editar, as mudanças só entrarão em vigor na próxima vez que o terminal for aberto, pois o `~/ .bashrc` já foi lido no início da sessão atual. Para aplicar as mudanças sem fechar o terminal, usamos o `source`:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ source ~/.bashrc
```

O nome `source` vem do inglês e significa "fonte". O comando funciona exatamente como o nome sugere: ele vai até o arquivo indicado, usa-o como fonte de comandos e os executa um por um no shell atual, como se você os tivesse digitado manualmente. Após executá-lo, todas as configurações entram em vigor imediatamente, sem precisar fechar o terminal:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ projeto
configs          erros.txt          log_completo.txt  scripts
data_atual.txt   estatisticas.txt   logs              testes
docs             lista_configs2.txt relatorio.txt      tmp.txt
erros2.txt        lista_configs.txt  resultados.txt
erros_encontrados.txt lista_scripts.txt  resultado.txt
```

A partir daí, todos os aliases, variáveis e o PATH configurados no `~/ .bashrc` estarão disponíveis em qualquer diretório do sistema, em todas as sessões futuras do terminal.